



Tecnológico de Monterrey

**Instituto Tecnológico y de Estudios Superiores de
Monterrey**

**Evidencia 1 - Actividad Integral de Conceptos Básicos y
Algoritmos Fundamentales (Evidencia Competencia)**

Materia: Programación de estructuras de datos y algoritmos fundamentales (Gpo
607)

Profesor: David Alonso Cantú Delgado

Por:

Isis Krystal Agramon Leal - A00843802

20 de septiembre del 2026

1. Plan de pruebas:

Para revisar que el programa funcionara correctamente, probé los siete algoritmos de ordenamiento con los dos archivos proporcionados y comparé los tiempos que tardaba cada uno. También hice pruebas con diferentes rangos de fechas para comprobar que la búsqueda funcionara aunque la fecha exacta no estuviera en los registros. Además, probé casos como rangos vacíos, fechas repetidas y datos con formato incorrecto para ver cómo respondía el programa.

2. Pruebas de algoritmos de ordenamiento:

Se ejecutó cada algoritmo con los dos archivos para comparar cómo cambiaba su tiempo dependiendo de qué tan ordenados estaban los datos.

Algoritmo	log607-1	log607-2
Swap Sort	5295.51 ms	530.862 ms
Bubble Sort	5263.75 ms	14.848 ms
Selection Sort	505.436 ms	504.287 ms
Insertion Sort	4799.05 ms	12.0827 ms
Quick Sort	22.8603 ms	152.107 ms
Merge Sort	55.3929 ms	49.1364 ms
Shell Sort	40.998 ms	29.6453 ms

3. Pruebas de búsqueda por rango:

Después de ordenar los registros, hice diferentes pruebas con la búsqueda por rango para revisar que se tomaran correctamente la fecha inicial y la fecha final. También probé casos especiales para asegurarme de que no se perdieran registros.

Prueba 1. Rango con fechas que no existen exactamente.

Se buscó desde Sep 8 2024 04:30:00 hasta Sep 10 2024 18:30:00. Aunque esas horas no aparecen exactamente en los registros, el programa encontró el primer registro después de la fecha inicial y tomó todos los registros que estaban dentro del rango.

```
Ingresa la fecha y hora inicial.
Usa el mes abreviado en ingles (Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec).
Formato: Sep 8 2024 04:30:00
Fecha inicial: Sep 8 2024 04:30:00
Ingresa la fecha y hora final.
Fecha final: Sep 10 2024 18:30:00
Key inicial: 20240908043000
Key final: 20240910183000
Posicion inicial encontrada: 1
Posicion final encontrada: 17
Deseas realizar otra ejecucion? Escribe s o n: n
```

Prueba 2. Rango vacío.

También probé un rango en el que no había registros, usando desde Jan 1 2030 00:00:00 hasta Jan 2 2030 00:00:00. En este caso, las dos posiciones obtenidas fueron 6818 y el programa mostró que no se encontraron logs en ese rango. Además, range607.txt quedó vacío para evitar que se conservaran resultados de una búsqueda anterior.

```
Usa el mes abreviado en ingles (Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec).
Formato: Sep 8 2024 04:30:00
Fecha inicial: Jan 1 2030 00:00:00
Ingresa la fecha y hora final.
Fecha final: Jan 2 2030 00:00:00
Key inicial: 20300101000000
Key final: 20300102000000
Posicion inicial encontrada: 6818
Posicion final encontrada: 6818
No se encontraron logs en ese rango.
Deseas realizar otra ejecucion? Escribe s o n: n
```

Prueba 3. Registros con la misma fecha y hora.

Para comprobar qué pasaba cuando había registros con la misma fecha y hora, utilicé el segundo archivo y busqué únicamente Feb 17 2026 20:05:15. El programa encontró dos registros diferentes con ese mismo timestamp. La posición inicial fue 4945 y la posición final 4947, por lo que los dos registros quedaron incluidos en range607.txt. Con esto comprobé que la búsqueda no elimina ni deja fuera registros cuando tienen la misma fecha y hora.

```
Ingresa la fecha y hora inicial.
Usa el mes abreviado en ingles (Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec).
Formato: Sep 8 2024 04:30:00
Fecha inicial: Feb 17 2026 20:05:15
Ingresa la fecha y hora final.
Fecha final: Feb 17 2026 20:05:15
Key inicial: 20260217200515
Key final: 20260217200515
Posicion inicial encontrada: 4945
Posicion final encontrada: 4947
Deseas realizar otra ejecucion? Escribe s o n: n
```

```
evidencia.cpp 1 resultados.txt M range607.txt M output608.txt M evidencia.exe C Sorts.h C Log.h C pointers.cpp C Fraction.h 5 C Node.h C test_node.cpp Act-1.3.1-Notación Asintótica
Evidences > Evidences1 > range607.txt
1 Feb 17 2026 20:05:15 10.14.50.213 Anomalous User Activity
2 Feb 17 2026 20:05:15 10.14.37.206 Security breaches
3
```

Prueba 4. Formato de fecha inválido.

También ingresé una fecha con un formato diferente al esperado, escribiendo September en lugar de Sep. El programa detectó que el mes no era válido y mostró un mensaje de error en lugar de continuar con la búsqueda. Con esta prueba comprobé que el programa puede detectar entradas que no siguen el formato establecido.

```
Peor caso: O(n^2)
Lista ordenada: 1
Primer log ordenado: 20240908002243
Ultimo log ordenado: 20260907235040
Ingresa la fecha y hora inicial.
Usa el mes abreviado en ingles (Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec).
Formato: Sep 8 2024 04:30:00
Fecha inicial: September 8 2024 04:30:00
Ingresa la fecha y hora final.
Fecha final: Sep 10 2024 18:30:00
Error: formato de fecha invalido.
```

Prueba 5. Límites de fechas.

Para probar los límites de las fechas, utilicé como inicio la fecha del primer registro y como final la del último registro. El objetivo fue comprobar que el programa pudiera incluir todos los datos cuando el rango abarcaba completamente el archivo. En este caso se deben obtener los 6818 registros.

```
Ingresa la fecha y hora inicial.  
Usa el mes abreviado en Ingles (Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec).  
Formato: Sep 8 2024 04:30:00  
Fecha inicial: Sep 8 2024 00:22:43  
Ingresa la fecha y hora final.  
Fecha final: Sep 7 2026 23:50:40  
Key inicial: 20240908002243  
Key final: 20260907235040  
Posicion inicial encontrada: 0  
Posicion final encontrada: 6818  
Deseas realizar otra ejecucion? Escribe s o n: n
```

4. Archivos generados:

Después de realizar el ordenamiento, el programa genera output608.txt con todos los registros ordenados cronológicamente. También genera range607.txt, donde se guardan únicamente los registros que se encuentran dentro del rango de fechas seleccionado.

Fragmento de output608.txt

```
evidencia.cpp 1 resultados.txt M range607.txt M output608.txt M evidencia.exe C Sorts.h C Log.h C pointers.cpp C Fraction.h 5 C Node.h C  
Evidences > Evidence1 > output608.txt  
1 Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication  
2 Sep 8 2024 04:37:37 10.14.248.79 Configuration errors  
3 Sep 8 2024 04:44:15 10.14.135.12 Application error  
4 Sep 8 2024 05:50:57 10.14.22.235 Changes in File Integrity  
5 Sep 8 2024 10:42:42 10.14.40.195 Data corruption  
6 Sep 8 2024 10:47:52 10.14.24.223 Data corruption  
7 Sep 8 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords  
8 Sep 8 2024 15:25:34 10.14.220.55 Broken authentication and session management  
9 Sep 9 2024 01:51:20 10.14.27.106 Application error  
10 Sep 9 2024 04:53:57 10.14.96.200 Suspicious IP Addresses  
11 Sep 9 2024 07:53:44 10.14.230.164 Data Exfiltration Attempts  
12 Sep 9 2024 15:18:08 10.14.182.26 Suspicious IP Addresses  
13 Sep 9 2024 18:59:55 10.14.132.80 Failed Login Attempts  
14 Sep 9 2024 19:17:57 10.14.186.180 Cross-Site Scripting  
15 Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses  
16 Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity  
17 Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity  
18 Sep 10 2024 21:18:36 10.14.183.213 Data Exfiltration Attempts  
19 Sep 11 2024 06:44:21 10.14.80.172 Suspicious IP Addresses  
20 Sep 11 2024 08:48:35 10.14.45.122 Anomalous User Activity  
21 Sep 11 2024 13:23:41 10.14.241.4 Changes in File Integrity  
22 Sep 11 2024 17:10:13 10.14.138.230 Encryption Failures
```

Fragmento de range607.txt

```
evidencia.cpp 1 resultados.txt M range607.txt M output608.txt M evidencia.exe C Sorts.h C Log.h C pointers.cpp C Fraction.h 5 C Node.h C  
Evidences > Evidence1 > range607.txt  
1 Sep 8 2024 04:37:37 10.14.248.79 Configuration errors  
2 Sep 8 2024 04:44:15 10.14.135.12 Application error  
3 Sep 8 2024 05:50:57 10.14.22.235 Changes in File Integrity  
4 Sep 8 2024 10:42:42 10.14.40.195 Data corruption  
5 Sep 8 2024 10:47:52 10.14.24.223 Data corruption  
6 Sep 8 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords  
7 Sep 8 2024 15:25:34 10.14.220.55 Broken authentication and session management  
8 Sep 9 2024 01:51:20 10.14.27.106 Application error  
9 Sep 9 2024 04:53:57 10.14.96.200 Suspicious IP Addresses  
10 Sep 9 2024 07:53:44 10.14.230.164 Data Exfiltration Attempts  
11 Sep 9 2024 15:18:08 10.14.182.26 Suspicious IP Addresses  
12 Sep 9 2024 18:59:55 10.14.132.80 Failed Login Attempts  
13 Sep 9 2024 19:17:57 10.14.186.180 Cross-Site Scripting  
14 Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses  
15 Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity  
16 Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity  
17
```

5. Conclusión:

Después de realizar las pruebas, pude ver que el orden inicial de los datos sí afecta bastante el tiempo de algunos algoritmos. Esto se notó principalmente con Insertion Sort y Bubble Sort, ya que fueron mucho más rápidos con el archivo que estaba casi ordenado. En cambio, Selection Sort tuvo tiempos muy parecidos con los dos archivos, porque sigue haciendo prácticamente las mismas comparaciones aunque los datos ya tengan cierto orden.

También me llamó la atención Quick Sort, porque fue más rápido con el archivo desordenado que con el archivo casi ordenado. Esto se relaciona con la forma en la que se

eligió el pivote en el algoritmo, ya que se utiliza el último elemento y un archivo casi ordenado puede hacer que las divisiones no sean tan equilibradas.

Con las pruebas de búsqueda también comprobé que se pueden utilizar fechas que no existan exactamente en los registros y que los registros repetidos no se pierden. Los casos de rango vacío y formato incorrecto también ayudaron a comprobar que el programa responde correctamente ante diferentes entradas.